

Corso di Programmazione Web

Modalità di esame AA 2025/2026

Docenti: Pierpaolo Loreti, Lorenzo Bracciale

Struttura dell'esame

L'esame prevede:

1. **Test a risposta multipla**
2. **Presentazione del progetto e discussione orale**

Il voto terrà conto di tre aspetti:

- qualità tecnica del progetto
 - capacità di spiegare e discutere le scelte adottate
 - comprensione degli argomenti del corso
-

Uso dell'intelligenza artificiale

L'uso di strumenti AI (ChatGPT, Claude, Copilot, etc.) è **esplicitamente consentito** e incoraggiato come parte del processo di sviluppo. Tuttavia, il progetto viene valutato anche sulla capacità dello studente di:

- **capire** il codice prodotto o suggerito dall'AI
- **spiegare** ogni parte del progetto durante l'interrogazione
- **modificare** il codice in modo consapevole, anche durante l'esame
- **mostrare** le specifiche o i prompt scritti per guidare l'AI

Presentare codice che non si è in grado di spiegare è equivalente a copiare: verrà valutato negativamente.

Il progetto

Tema

Il tema è libero. L'applicazione deve essere un'applicazione web completa con frontend e backend, che permetta di mostrare chiaramente le competenze viste nel corso.

Esempi di temi:

- catalogo di libri, film, giochi, ricette o prodotti
- lista di attività o promemoria
- gestione di prenotazioni o iscrizioni
- raccolta di recensioni
- piccola dashboard con dati inseriti e visualizzati dall'utente
- applicazione didattica o quiz

Requisiti tecnici obbligatori

Il progetto deve includere:

Frontend (browser)

- struttura HTML semantica
- layout CSS leggibile e responsive (Flexbox e/o Grid)
- codice JavaScript separato dall'HTML
- manipolazione del DOM
- gestione di eventi utente (click, submit, input)
- chiamate HTTP al backend tramite `fetch` con `async/await`
- gestione visibile degli stati: caricamento, errore, successo, lista vuota
- validazione lato client dei dati inseriti dall'utente

Backend (Node.js + Express)

- progetto inizializzato con `package.json`
- server Express con almeno le seguenti rotte REST:
 - `GET /api/risorsa` — restituisce una lista
 - `GET /api/risorsa/:id` — restituisce un elemento
 - `POST /api/risorsa` — crea un nuovo elemento
- risposte in formato JSON con codici di stato HTTP appropriati
- validazione lato server dei dati ricevuti (es. nel `POST`), con risposta `400` in caso di dati mancanti o non validi
- gestione minima degli errori (404, 400, 500)
- middleware per il parsing del body JSON e per i file statici

Dati

- i dati possono essere gestiti in memoria, in un file JSON, o con un database leggero

Git

- repository GitHub Classroom con commit significativi e messaggi chiari (il link all'assignment verrà fornito dal docente)

- la storia dei commit deve mostrare le fasi di sviluppo (non un unico commit finale)
-

Cosa mostrare all'esame

Durante la presentazione e l'interrogazione, lo studente deve essere in grado di mostrare:

1. Il funzionamento dell'applicazione

Dimostrare le funzionalità principali nel browser.

2. Il codice – comprensione e spiegazione

Spiegare le scelte architetturali: perché quella struttura, perché quelle rotte, come funziona quel componente. Il docente potrà chiedere di modificare o estendere una parte del codice al momento.

3. Strumenti di analisi e debug

Mostrare come si usa l'applicazione dall'interno:

- **DevTools del browser:** pannello Elements, Console, Network, Sources
- **Ispezione delle richieste HTTP:** headers, payload, response, status code
- **Log lato server:** messaggi di log nel terminale Node.js
- **Debugging:** breakpoint nel browser o con Node.js inspector (`--inspect`)

4. Test funzionali

Mostrare gli scenari testati manualmente: input valido, input non valido, lista vuota, errore di rete simulato. Lo studente deve saper descrivere cosa si aspettava e cosa ha verificato.

5. Git – storia dello sviluppo

Mostrare la storia dei commit su GitHub o con `git log`. La sequenza deve essere leggibile e deve riflettere un processo di sviluppo progressivo.

6. Specifiche scritte per l'AI

Se lo studente ha usato strumenti AI, deve mostrare:

- i prompt o le specifiche scritte per guidare la generazione del codice
 - le modifiche apportate al codice generato
 - le parti scritte autonomamente
-

Documento di specifiche (da consegnare prima dello sviluppo)

Prima di iniziare lo sviluppo, ogni studente deve caricare nel repository un breve documento di specifiche. La consegna avviene tramite **commit e push** con messaggio standard:

specifiche: primo documento di specifica del progetto

Il documento deve essere in formato Markdown (`.md`) nella cartella `specifiche/` e deve descrivere:

1. **Obiettivo dell'applicazione:** a cosa serve, che problema risolve, a chi è rivolta
2. **Cosa può fare l'utente:** le azioni principali che il sistema rende possibili, descritte dal punto di vista dell'utente (non delle rotte o del codice)
3. **Struttura del frontend:** pagine o sezioni, cosa vede e cosa può fare l'utente in ciascuna
4. **Mockup minimale:** schizzi o wireframe dell'interfaccia (anche disegnati a mano e fotografati)
5. **Scenari di test:** situazioni concrete che si intendono verificare (cosa si fa, cosa ci si aspetta)

Il documento di specifiche può essere scritto anche con l'aiuto dell'AI, ma deve essere discusso all'esame.

README.md obbligatorio

Il repository deve contenere un `README.md` con:

- titolo e descrizione dell'applicazione
- istruzioni per installare le dipendenze (`npm install`)
- istruzioni per avviare backend e frontend
- elenco delle funzionalità implementate
- elenco delle funzionalità extra (se presenti)
- note sull'uso dell'AI (strumenti usati, parti generate, parti modificate)

Argomenti del corso da mostrare nel progetto

Questa è la parte centrale della valutazione. Il progetto è il mezzo con cui lo studente dimostra di aver compreso e saputo applicare gli argomenti visti a lezione. All'esame il docente verificherà la padronanza di ciascuno di questi temi, sia attraverso il codice sia attraverso le domande. Non è necessario usare ogni singola tecnica, ma lo studente deve saper riconoscere, spiegare e all'occorrenza modificare le parti del progetto che riguardano ognuno dei punti seguenti.

Git e GitHub — uso del repository, commit con messaggi chiari, storia di sviluppo leggibile, branch e gestione delle modifiche, README come documentazione del progetto.

Internet, Web e HTTP — funzionamento client/server, struttura di una URL, ciclo richiesta/risposta, metodi (GET, POST e gli altri), status code, header, formato JSON, e il significato di CORS (anche se non sempre necessario nel progetto).

HTML — struttura semantica del documento, organizzazione dei contenuti, form e input, attributi, accessibilità di base.

CSS — selettori e specificità, box model, layout con Flexbox e Grid, design responsive, organizzazione e leggibilità dei fogli di stile.

JavaScript — manipolazione del DOM, gestione degli eventi utente, programmazione asincrona con Promise, chiamate `fetch` al backend, uso di `async/await`, gestione degli errori e degli stati dell'interfaccia.

Node.js — uso di npm e `package.json`, sistema dei moduli, avvio e gestione del server, lettura dei log dal terminale.

Express — routing, middleware, parametri di rotta e query, parsing del body, servire file statici.

REST — progettazione delle rotte attorno alle risorse, scelta dei metodi e degli status code, struttura coerente delle risposte JSON.

Le slide del corso (cartella `pdf`) costituiscono il riferimento per tutti questi argomenti.

Funzionalità extra (facoltative)

Chi completa i requisiti minimi può aggiungere:

- modifica e cancellazione degli elementi (PUT/DELETE)
- ricerca o filtro lato server
- persistenza su file JSON o database (SQLite, MongoDB)
- autenticazione semplice (sessioni o token)
- test automatici con `jest` e `supertest`
- deploy online (Render, Railway, Vercel, etc.)
- uso di EJS o altro template engine lato server

Le funzionalità extra devono essere funzionanti e documentate.